ACADEMIC PROJECT REPORT
ON
PORT DIGITAL CLEARANCE SYSTEM
FOR NMPA MANGALURU

Submitted in partial fulfillment of the requirements for the degree of
Master of Computer Applications (MCA)

DEPARTMENT OF COMPUTER APPLICATIONS
UNIVERSITY OF MANGALURU
Submission Year: 2026

TABLE OF CONTENTS

Chapter 1: Synopsis

Introduction

Vessel clearance at major ports like New Mangalore is still a slow, manual process. During our project research, we found that ship agents must visit multiple separate departments to get permission for a vessel to enter or leave the port. PHO (Port Health Organisation) handles health checks. Customs looks after duties. Traffic Control manages the actual berthing slots. In the old system, the agent must run around with physical papers, like manifests and crew profiles, to get stamps from different offices. If one officer is away, the vessel just sits idle at the outer anchorage buoy. This delays cargo handling and costs shipping lines thousands of dollars in daily demurrage. We developed this web application to solve this problem by creating a single-window portal where agents file clearance requests digitally.

1.1. Introduction of the System

1.1.1. Project Title

We titled our project "Port Digital Clearance System for NMPA Mangaluru" (National Maritime Single Window Portal).

1.1.2. Category

We categorized this project as an Enterprise Web Application and Portal. We developed it using the MERN stack (MongoDB, Express, React, Node.js). We also built a local storage mock backend capability so that users can test and run the application during offline scenarios.

1.1.3. Overview

We designed this web portal for the New Mangalore Port Authority (NMPA) at Panambur. We chose the MERN stack: MongoDB, Express, React, and Node.js. Ship agents can register vessels, upload documents, and apply for voyages. Our system then routes the application to the Port Health Officer (PHO), Customs, and Traffic Control in a strict sequential order. We built custom dashboards for each officer role to review and approve or reject requests. Once cleared by all departments, the portal generates a bilingual PDF certificate with a secure QR code. We also

added an offline mock backend using React's mock adapter and LocalStorage, which keeps the frontend working even if the port loses internet connectivity.

1.2. Background

1.2.1. Introduction of the Company

The New Mangalore Port Authority (NMPA) is located at Panambur, Mangaluru. It is one of the major ports in India. The port handles a massive amount of cargo daily, including LPG, crude oil, coal, liquid chemicals, and containers. NMPA functions under the Ministry of Ports, Shipping and Waterways. Currently, clearing a vessel requires multiple physical handovers of documents. In this MCA project, we designed and built a single window system to digitize this entire work. Our goal is to eliminate administrative delays and make port approvals completely paperless.

1.3. Objectives of the System

In this project, we focused on the following main objectives:

- Cut down vessel waiting times at NMPA by digitizing manual approvals.
- Secure the login module using role-based access and Google Authenticator 2FA.
- Generate official documents in both Hindi and English as per government rules.
- Build a local storage offline backup so agents can work even with poor network connections.

1.4. Scope of the System

We defined the scope of our system to cover user registration with admin approval, a vessel registry for IMO details, a voyage filing workflow, and a PDF generator. We also built pilot verification pages and a read-only MongoDB database to log all actions for audit purposes.

1.5. Structure of the System

We built the application with a React frontend and a Node.js/Express backend. We stored data in a MongoDB cloud cluster. If the port network goes down, we designed a custom mock adapter that intercepts API calls and reads/writes to the browser's LocalStorage. This keeps the UI completely active.

1.6. System Architecture

The three-tier architecture consists of the presentation tier, the application routing tier, and the database storage tier. Below is the system architecture layout:

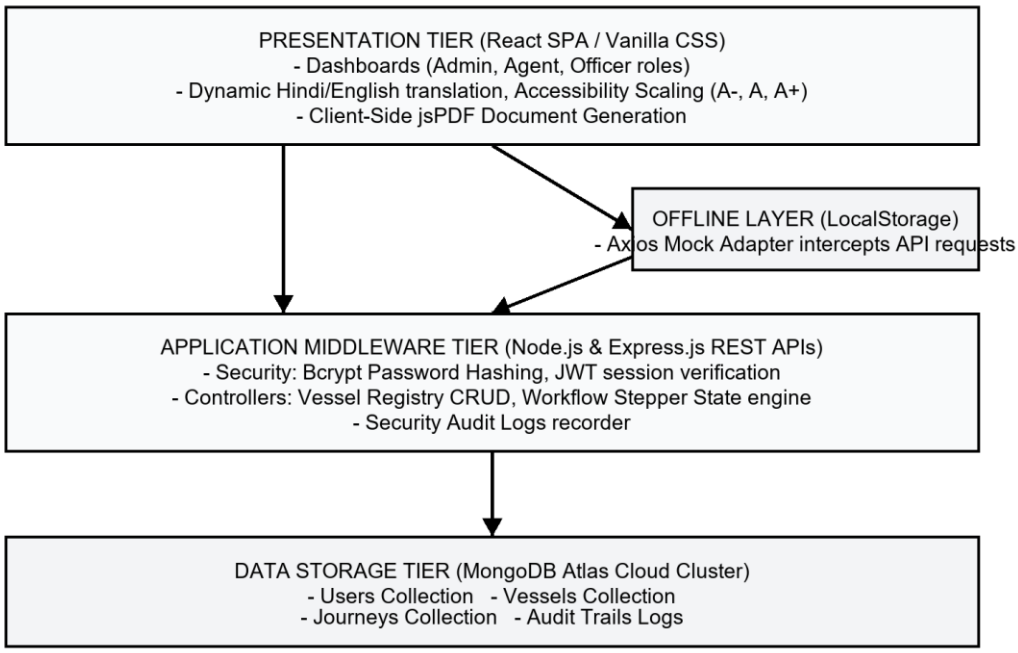

Figure 1.1: System Architecture Diagram

1.7. End Users

The portal is designed for five types of users:

1. Ship Agent: Files vessel entries, registers cargo/passenger lists, and prints approved clearances.
2. Port Health Officer (PHO): Inspects vessel medical logs and declares maritime health clearances.
3. Customs Department: Audits lighthouse dues payments and checks import/export cargo declarations.
4. Port Traffic Control: Assigns anchorage coordinates, pilot services, and issues the final clearance to dock.
5. System Administrator: Supervises user registrations, resets 2FA keys, and exports security audits.

1.8. Software/Hardware used for the development

Development environment details:

- Operating System: Windows 11 Professional
- Code Editor: Visual Studio Code v1.85
- Runtime Platform: Node.js v20.10.0 & npm v10.2.3
- Database Management: MongoDB Community Server v7.0 & MongoDB Compass
- Development RAM: 16 GB DDR4
- Processor: Intel Core i5 (11th Gen, 4 Cores, 2.7 GHz)

1.9. Software/Hardware required for the implementation

Implementation specifications for production:

- Server Hosting Environment: Linux VPS (Ubuntu Server 22.04 LTS), 2 vCPU, 4GB RAM, 40GB SSD
- Database Hosting: MongoDB Atlas Cloud Tier (M10 cluster or higher)
- Security Layer: SSL/TLS certificates (HTTPS port 443)
- Client Specifications: Modern web browser (Google Chrome 90+, Microsoft Edge, or Mozilla Firefox)
- Authenticator App: Smart mobile device running Google Authenticator or Microsoft Authenticator app

Chapter 2: SRS

2.1. Introduction

In this SRS document, we list all the features and requirements for our NMPA Port Digital Clearance System. We define how ship agents and port officers interact with the system.

2.2. Overall Description

2.2.1. Product perspective

Our software links ship agents directly with the port health, customs, and traffic offices. We implemented a sequential approval chain that prevents vessels from docking without proper clearance.

2.2.2. Product Functions

The portal supports:

- Role-based login and signup with admin control.
- Registration of commercial vessels with IMO number verification.
- Step-by-step voyage application and review tracking.
- Automated bilingual PDF certificate generation with QR codes.
- QR code verification route for boarding pilots.
- Read-only security audit log trail.

2.2.3. User characteristics

The users of our system are professional port personnel. Port Health Officers and Customs Officers generally have basic web browser familiarity. We assume System Administrators possess a technical background to manage security logs, seed databases, and configure TOTP keys.

2.2.4. General constraints

- The client interface forces session logout after 160 seconds of inactivity to protect shared terminal screens.

- Offline mode is limited to 5MB of browser LocalStorage.
- Audit logs are read-only and cannot be updated or deleted by any user.

2.2.5. Assumptions

- Users possess mobile devices to scan QR codes and display authenticator tokens.
- The port offices maintain network connections to synchronize MongoDB records.

2.3. Special Requirements

We designed the system to comply with official government portals. We implemented a tricolor layout, bilingual translation maps between English and Hindi, and font scaling controls (A-, A, A+) to support accessibility.

2.4. Functional requirements

- FR-1 User Onboarding: New users register and wait in a pending list. The admin must verify them. Once approved, the system shows a 2FA QR code.
- FR-2 Vessel Registry: Agents must enter the vessel's name, unique 7-digit IMO number, flag state, and tonnage. The system rejects duplicate IMO numbers.
- FR-3 Voyage Clearance: Agents apply for voyages. The request goes through three steps: PHO check, Customs check, and Traffic Control berthing check. The status changes to Cleared only after all three approve.
- FR-4 PDF Certification: The system creates a bilingual PDF certificate with a verification QR code.
- FR-5 Verification Route: Scanning the QR code opens a public web page showing the voyage's clearance status.
- FR-6 Audit Trails: The backend writes all approvals and login details into a secure MongoDB audit log collection.

2.5. Design Constraints

We built the frontend using React and pure CSS without Tailwind. We secured communications using JSON Web Tokens (JWT) inside HTTP authorization headers. We hash user passwords using bcrypt before database storage.

2.6. System Attributes

- Security: Passwords are encrypted with bcrypt. Active sessions require JWT verification. TOTP checks are required during login.
- Performance: Backend database queries and JSON API replies should process in under 500 milliseconds.
- Reliability: The system implements an offline adapter fallback to local storage if the database server is unreachable.

2.7. Other Requirements

We implemented a client-side database polling daemon that updates the clearance dashboards every 5 to 6 seconds. This ensures that status transitions are immediately reflected on officer consoles without manual web browser reloads.

Chapter 3: System Design

3.1. Introduction

In this chapter, we cover the overall design of our application. We include the context flow, program structures, and DFD models for different operations.

3.2. Assumptions and Constraints

We note that data synchronization is limited by local storage capacity when offline. We configured our asynchronous polling daemon to run every 6 seconds to balance UI updates with database workload.

3.3. Functional decomposition

We decomposed the system features into a hierarchical structure mapping Identity Provisioning, Voyage Clearances, and System Utilities. I designed the diagram below to illustrate this functional architecture:

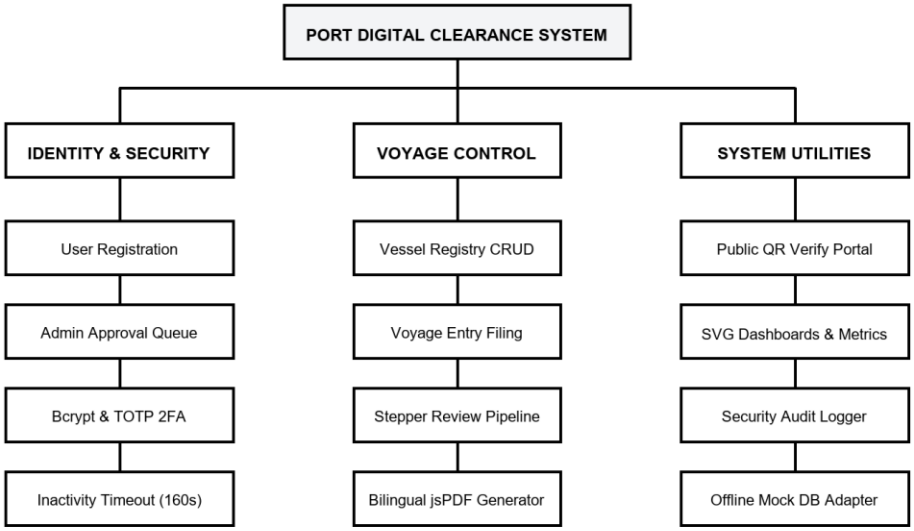

Figure 3.1: Functional Decomposition Diagram

3.4. Description of Programs

3.4.1. Context Flow Diagram (CFD)

The Context Flow Diagram to outline data exchanges between our core portal and external actors:

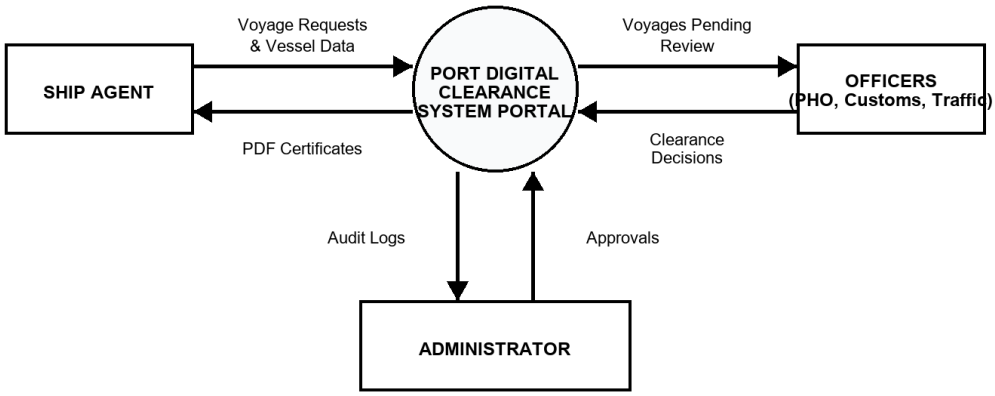

Figure 3.2: Context Flow Diagram (CFD)

3.4.2. Data Flow Diagrams (DFDs - Level 0, Level 1, Level 2)

The Below DFD Level 0 to represent high-level login processing and credentials verification:

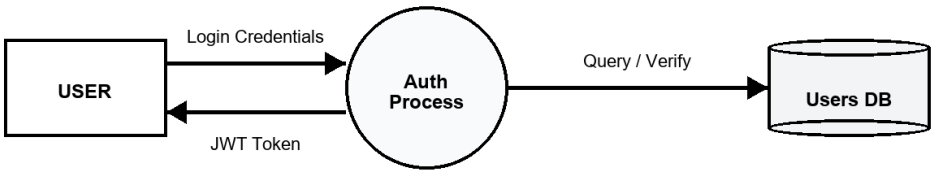

Figure 3.3: Data Flow Diagram (DFD) Level 0

The Below DFD Level 1 to represent the primary logical components of our single window application:

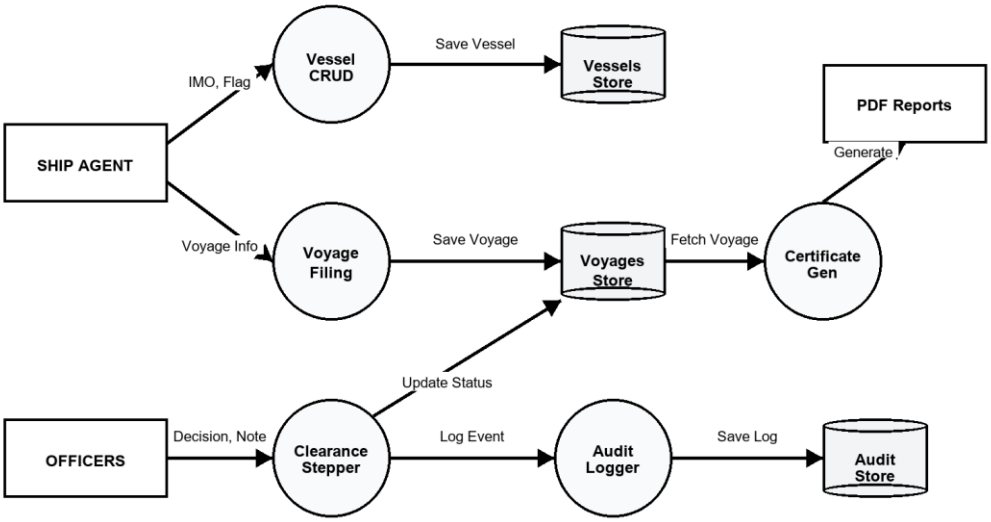

Figure 3.4: Data Flow Diagram (DFD) Level 1

The below DFD Level 2 shows detail the state transition steps in our clearance stepper:

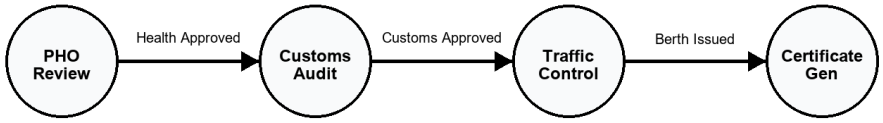

Figure 3.5: Data Flow Diagram (DFD) Level 2 - Stepper Flow

Chapter 4: Database Design

4.1. Introduction

We selected MongoDB Atlas for our database storage. In this chapter, I define our collection schemas and the relationships between them.

4.2. Purpose and scope

We configured our database to store records for user credentials, vessel registries, voyage clearances, and security audits. I set up database constraints to maintain data integrity across collections.

4.3. Table Definitions

User Collection Schema (Users)

Field Name	Data Type	Constraint	Description
username	String	Required, Unique	Account handle name
password	String	Required	Bcrypt hashed password
email	String	Required	User email address
role	String	Required	Administrator, Agent, PHO, Customs, Traffic
status	String	Default: pending	pending, approved, rejected
twoFactorSecret	String	Optional	Base32 secret key for TOTP app
is2FAEnabled	Boolean	Default: true	Two-factor enforcement flag

Vessel Collection Schema (Vessels)

Field Name	Data Type	Constraint	Description
name	String	Required	Name of registered vessel
imoNumber	String	Required, Unique	Unique 7-digit IMO registry key
flagState	String	Required	Country of registration flag
vesselType	String	Required	Cargo, LPG, Tanker, Container, Passenger
ownerDetails	String	Required	Shipping line owner description
grt	Number	Optional	Gross Registered Tonnage value
nrt	Number	Optional	Net Registered Tonnage value
userId	String	Optional	Creator agent account ID reference

Voyage Collection Schema (Journeys)

Field Name	Data Type	Constraint	Description
vessel	Object	Required	Snapshot of vessel metadata document

lastPortOfCall	String	Required	Last origin port of voyage
eta	Date	Required	Estimated Time of Arrival
etd	Date	Required	Estimated Time of Departure
status	String	Default: In Progress	In Progress, Cleared, Rejected
clearances	Object	Default: Pending	Sub-fields: health, customs, traffic statuses
notes	Object	Default: Empty	Sub-fields: health, customs, traffic review comments
captainName	String	Optional	Master of the vessel
destinationPort	String	Optional	Destination port code
cargoType	String	Default: BALLAST	BALLAST, CONTAINER, CRUDE, LPG, LNG
crewCount	Number	Default: 0	Number of crew members on board
passengerCount	Number	Default: 0	Number of passengers on board
ilhReceiptNo	String	Optional	Lighthouse dues receipt number

ilhPaidDate	Date	Optional	Date of lighthouse dues payment
ilhAmount	Number	Default: 0	Lighthouse dues amount paid in INR
ilhValidFrom	Date	Optional	Lighthouse validity start date
ilhValidTo	Date	Optional	Lighthouse validity expiry date
healthCertificateNo	String	Optional	Generated health certificate reference
portClearanceNo	String	Optional	Generated port clearance reference
userId	String	Optional	Creator agent user ID reference

Audit Trail Collection Schema (AuditTrails)

Field Name	Data Type	Constraint	Description
timestamp	Date	Default: Date.now	Creation date and time of record
user	String	Required	Username executing the action
action	String	Required	Description of system event logged

4.4. ER Diagram

The Entity-Relationship diagram details relations between users, vessel registries, voyages, and audit trails. Users in agent roles enroll multiple vessels (1:N) and file multiple journeys (1:N). Voyages embed technical snapshots of vessel records to ensure historical data consistency.

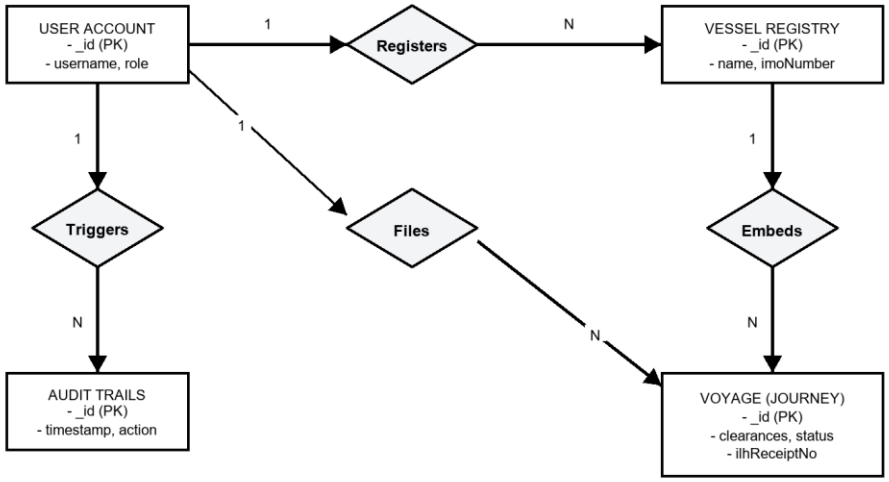

Figure 4.1: Entity Relationship (ER) Diagram

Chapter 5: Detailed Design

5.1. Introduction

In this chapter, I have described the low-level structure of our codebase and detail the modules we built.

5.2. Structure of the software package

The project is structured as a decoupled web package. Below is the file directory layout:

```
mca_project_remote/
├── backend/
│   ├── server.js (Main Express app entry)
│   ├── seed.js (Pre-populated role accounts script)
│   ├── models/ (Mongoose collections schemas)
│   │   ├── User.js
│   │   ├── Vessel.js
│   │   ├── Journey.js
│   │   └── AuditTrail.js
│   └── routes/
│       └── (REST controller API handlers)
└── frontend/
    ├── index.html
    └── src/
        ├── main.jsx (React initialization)
        ├── App.jsx (Navigation Router, Layout, Translation maps)
        ├── App.css (Global theme and styling)
        ├── mockBackend.js (LocalStorage offline database mock adapter)
        ├── Login.jsx (Credentials console and 2FA prompt)
        ├── VesselRegistry.jsx (Technical registrations CRUD)
        ├── ClearanceWorkflow.jsx (Clearance stepper forms & lists)
        ├── Dashboard.jsx (SVG charts and weather indicators)
        ├── AdminPanel.jsx (User approval queue)
        └── VerifyCertificate.jsx (Public QR verification router)
```

5.3. Modular decomposition of the System

5.3.1. Module 1: Authentication and Session Security

This security module manages logins and enforces credentials checks across user profiles.

5.3.1.1. Inputs

The login security console expects the following fields:

- Username (String, alphanumeric credentials)
- Password (String, plain-text security code)
- TOTP security code (6-digit dynamic number from Google Authenticator)

5.3.1.2. Procedural details

1. I have configured the API to check the database for the entered username.

2. If found, our backend compares the password with the saved bcrypt hash.

3. The server verifies that the user status is set to approved.

4. If 2FA is active, our code compares the user's TOTP input with the calculated server passcode.

5. On success, the API signs a JWT session token for the user.

5.3.1.3. File I/O interface

Reads: Our code reads from the User collection to retrieve hashed credentials and 2FA secrets.

Writes: Our system writes to the Audit Trail collection to record successful login events or failures.

5.3.1.4. Outputs

Outputs produced upon credentials verification:

- Successful case: JSON Web Token (JWT) session string, user role, and account username.
- Failure case: HTTP 401 (Invalid Credentials) or HTTP 403 (Account Pending Admin Approval) JSON response.

5.3.1.5. Implementation Aspects

We configured our React client to track mouse clicks and keystrokes. If a user is inactive for 160 seconds, our frontend automatically logs them out and clears their JWT token to prevent unauthorized access on shared desks.

5.3.2. Module 2: Vessel Registry and Workflow Controller

This module manages vessel registration data and coordinates the sequential approval pipeline across different port authorities.

5.3.2.1. Inputs

The vessel registry and voyage filing forms expect the following data points:

- Vessel Name (String, alphabetical name of the ship)
- IMO Number (7-digit unique International Maritime Organization number)
- Voyage details (Estimated Time of Arrival, cargo specifications, and captain details)

5.3.2.2. Procedural details

1. I have designed the vessel registry form to validate that the IMO number contains exactly 7 digits.

2. The server-side controller queries the MongoDB Vessels collection to prevent duplicate IMO registration.

3. Once the vessel is registered, the ship agent can submit a clearance request, initializing the stepper state to 'PHO Pending'.

4. Our workflow controller updates the clearance status step-by-step as each department officer approves the request.

5.3.2.3. File I/O interface

Reads: Our system reads from the Vessels database to verify duplicate entries and fetches voyage records to render the stepper timeline.

Writes: Our controller writes new vessel documents to the Vessels collection and updates clearance state transitions in the Journeys collection.

5.3.2.4. Outputs

Outputs produced during registry and workflow operations:

• Successful case: New vessel document created, and voyage stepper status updated on officer consoles.

• Failure case: Form validation warnings or duplicate IMO error messages displayed to the agent.

Chapter 6: User Interface

6.1. Login

I have designed the Login page with a clean tricolor interface. We allow users to login or switch to the signup form. If details match, our page asks for a 2FA code. First-time users see a QR code to sync their authenticator app.

6.2. Main Screen/Home Page

We designed the Home page with a tricolor header banner to represent national compliance guidelines. I included accessibility controls to resize fonts (A-, A, A+) and language buttons to translate the UI. The main panel adjusts dynamically to display the dashboard based on the user's role.

6.3. Menu

The navigation menu resides in a collapsable sidebar panel. It updates dynamically to match the role permissions of the logged-in user:

- Ship Agent: Renders links for Dashboard, Vessel Registry, and Clearance Workflow.
- Department Officers (PHO, Customs, Traffic): Renders links for Dashboard and Clearance Hub review queues.
- System Administrator: Renders links for Dashboard, User Approvals, and Security Audit Logs.

6.4. Data Store/Retrieval/Update

We set up forms to manage all data inputs. Our Vessel Registry form captures vessel parameters (name, flag state, IMO number, vessel type, gross tonnage, and net tonnage). Our Voyage Clearance form allows ship agents to enter arrival details, destination codes, crew lists, and lighthouse dues receipt numbers, paid amounts, and validity dates.

6.5. Validation

We set up data validation at both the client and database layers. I configured form components to run regex checks to ensure IMO numbers contain 7 digits, tonnages are numeric values, and dates are valid. Our database schema rejects duplicate IMO entries.

6.6. View

Our main view displays a structured voyage registry grid. I built this grid to show the vessel name, IMO number, captain details, arrival/departure dates, and overall clearance status. We color-coded the statuses: green for "Cleared", yellow for "In Progress", and red for "Rejected".

6.7. On Screen reports

The dashboard displays data using simple SVG charts:

- SVG Status Bar: Shows the number of cleared and pending voyages.
- Carbon Calculator: Estimates carbon emissions based on vessel cargo and distance.
- Port Weather Panel: Displays wind speeds and wave heights for the Mangaluru coastline.

6.8. Alerts

Our system triggers toast notifications on user consoles during clearance status transitions and form submissions. We also designed a simulated SMS Toast alert to mimic notifications sent to agents on approval, using masked phone details to protect data privacy.

6.9. Error message

We set up the system to display modal error alerts for operational issues: invalid login passcodes, duplicate IMO registrations, missing lighthouse dues receipts, or unauthorized API access attempts.

Chapter 7: Testing

7.1. Introduction

We ran three testing phases to validate functional requirements, security compliance, and UI responsiveness.

7.1.1. Unit Testing

I ran unit tests to validate core utility routines in isolation. We tested the bcrypt password hashing algorithm, evaluated TOTP generators against time drifts, checked date formatting utilities, and validated canvas-based Hindi text scaling ratios.

7.1.2. Integrate Testing

In our integration tests, we focused on the sequential clearance state machine. We verified that voyage records successfully transitioned through states: "PHO Pending" -> "Customs Pending" -> "Traffic Pending" -> "Cleared". We also verified that the final approval unlocked the PDF download buttons.

7.1.3. System Testing

During system testing, we evaluated the end-to-end user workflows: self-registration, admin approval, authenticator setup, vessel registry enrollment, voyage filing, officer review, PDF generation, and public QR certificate verification.

7.2. Test Reports

I have compiled the test execution log below detailing scenarios, inputs, expected behaviors, and outcomes:

Test Scenario	Input Details	Expected Behavior	Result
User Registration Approval	Role: PHO Officer, status pending	Access blocked until admin approval occurs.	**PASSED**
2FA Security Check	Username: Agent1, Invalid 6-digit TOTP	Block access, show "Invalid 2FA token" alert.	**PASSED**
Vessel IMO Uniqueness	Vessel: test1, IMO: 9012345 (Duplicate)	Database validation error; reject registration.	**PASSED**
Inactivity Logout	No cursor/keyboard events for 160 seconds	Log out user and redirect to login screen.	**PASSED**
Bilingual PDF Verification	Scan QR on Health certificate	Redirect browser to public verify route and fetch voyage data.	**PASSED**

Conclusion

We successfully digitized and automated the multi-department clearance pipeline at New Mangalore Port. By replacing manual paperwork with our Single Window Portal, we improved coordination, reduced vessel turnaround times, and eliminated administrative gaps. We used multi-factor authentication (2FA) and public QR code verification to secure operations, and built an offline local storage fallback to ensure the portal remains available at remote port facilities.

Limitations

- We note that offline mode is limited to 5MB of browser LocalStorage.
- I observed that dynamic client-side PDF compilation depends on device system fonts, which can occasionally cause formatting shifts.
- In our tests, synchronizing data updates required manual browser refreshes when operating in full offline fallback environments.

Scope for enhancement

- AIS Feeds Integration: We plan to connect the vessel registry to live Automatic Identification System (AIS) transponder feeds to track coordinates on interactive maps.
- Blockchain Auditing: We could record clearance histories on a blockchain ledger to provide secure, immutable logs for international maritime audits.
- EDIFACT Support: We can integrate international shipping message standard formats (such as EDIFACT CUSCAR) to support electronic data interchange.

Abbreviation and Acronyms

Abbreviation / Acronym	**Definition**
NMPA	New Mangalore Port Authority
SRS	Software Requirements Specification
MERN	MongoDB, Express.js, React, Node.js Stack
PHO	Port Health Organisation / Port Health Officer

CBIC	Central Board of Indirect Taxes and Customs
TOTP	Time-Based One-Time Password (2FA)
JWT	JSON Web Token
IMO	International Maritime Organization
GRT / NRT	Gross Registered Tonnage / Net Registered Tonnage
ILH	Indian Light House Dues
SPA	Single Page Application
CFD / DFD	Context Flow Diagram / Data Flow Diagram
ER Model	Entity-Relationship Model
CRUD	Create, Read, Update, Delete

Bibliography

1. Ministry of Ports, Shipping and Waterways, Government of India. Guidelines for National Maritime Single Window Portal, 2023.
2. World Health Organization (WHO). International Health Regulations (IHR 2005) - Second Edition.
3. Ministry of Health & Family Welfare, Government of India. Indian Port Health Rules, 1955.
4. Central Board of Indirect Taxes and Customs (CBIC). The Customs Act, 1962 (India).
5. Elmasri, R., & Navathe, S. B. Fundamentals of Database Systems, 7th Edition, Pearson.
6. jsPDF Library Community, Client-Side JavaScript PDF Compilation Reference, GitHub documentation.